

Data Mining Practice8 - 버리

Task1 Decision Tree 모델을 사용해서 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기

```
library(rpart)
train_data <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/9week practice/practice8/train.csv")
test_data <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/9week practice/practice8/test.csv")
sample_data <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/9week practice/practice8/sampleSubmission.csv")
```

데이터와 decision tree모델링을 위한 라이브러리를 임포트한다.

데이터의 예측을 위해서는 데이터의 형태 및 특성을 확인한 후, 각 변수별로 영향을 줄 요소들, 그리고 이를 조정하기 위해 파라미터를 수정하는 등의 작업이 필요하지만, 우선은 decision Tree를 모든 변수를 사용하여 예측을 수행해보고, 조금 더 특정 요소들에 가중치를 주는 방식으로 성능을 높여보고자 한다

```
# 3. datetime을 연, 월, 일, 시간으로 분리
train_data$datetime <- as.POSIXct(train_data$datetime)
train_data$year <- as.numeric(format(train_data$datetime, "%Y"))
train_data$month <- as.numeric(format(train_data$datetime, "%m"))
train_data$day <- as.numeric(format(train_data$datetime, "%d"))
```

```
d"))
train_data$hour <- as.numeric(format(train_data$datetime,
"%H"))

# test 데이터도 동일하게 처리
test_data$datetime <- as.POSIXct(test_data$datetime)
test_data$year <- as.numeric(format(test_data$datetime, "%Y"))
test_data$month <- as.numeric(format(test_data$datetime, "%m"))
test_data$day <- as.numeric(format(test_data$datetime, "%d"))
test_data$hour <- as.numeric(format(test_data$datetime, "%H"))
```

우선 데이터가 시계열 데이터이기 때문에 datetime이 하나로 연결되어 있으면 제대로 된 예측이 어려울 것이라 생각하여, 이를 분리하였다.

```
# 4. 불필요한 열 제거
train_data <- subset(train_data, select = -c(datetime, casual, registered))
X_train <- subset(train_data, select = -count)
y_train <- train_data$count
```

이후 train데이터에서 datetime이나 casual, registered와 같은 예측에 악영향을 줄 요소를 제거하였다.

```
# test 데이터는 count 열이 없으므로 그냥 전체 사용
X_test <- subset(test_data, select = -datetime)

# 5. Decision Tree 모델 학습
model <- rpart(count ~ ., data = train_data, method = "anova")
```

우선 Decision Tree 학습에 rpart를 사용해서 train데이터에 대해 학습을 진행하였다. method anova는 연속형 변수에 대해 예측하는 방법이고, 영향을 줄 변수는 아까 제외한 datetime이나 casual, registered 외 모든 변수를 예측에 사용하였다.

```
# 6. test 데이터에 대한 예측
y_pred_test <- predict(model, X_test)
sample_data$count <- y_pred_test

# sample_data를 "sample_submission.csv"로 저장
write.csv(sample_data, "predict dicisiontree1.csv", row.names = FALSE)
```

이제 학습된 모델을 가지고 test데이터에 대해서 예측을 수행하고, sample_data를 참고하여 count에 예측된 값을 입력한 후 predict dicisiontree1.csv파일로 저장하여, 이를 kaggle에 제출하였다.

제출 및 설명	개인 점수 ①	공개 점수 ①	선택된
 예측 dicisiontree1.csv 완료(마감일 이후) · 21초 전	0.91832	0.91832	☐

개인 점수가 0.91832로 나왔는데, 이 값이 RMSE값인지는 잘 모르겠다.

Task2 1번의 모델에 bagging을 적용하여 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기

이제, 지금 만든 decision tree 모델에서 변수를 변경하는 것으로 모델의 성능을 높이고 싶은데, 지금 만든 모델의 RMSE값을 모르기 때문에, 일단 Task2의 bagging을 적용해 보기로 하였다.

bagging을 위해

```
# 필요한 패키지 설치
install.packages("ipred")

# 패키지 로드
library(ipred)
```

다음 과정을 진행한다

```
# Bagging을 사용한 Decision Tree 모델 학습
bagging_model <- bagging(count ~ ., data = train_data, nbagg
g = 25) # nbagg: 트리의 개수
```

다음은 bagging을 사용해서 decision tree 모델을 학습한다.

```
# 테스트 데이터에 대한 예측
y_pred_test_bagging <- predict(bagging_model, test_data)

# 예측 결과를 sample_data의 count 열에 저장
sample_data$count <- y_pred_test_bagging

# CSV 파일로 저장
write.csv(sample_data, "predict_bagging_decisiantree1.csv",
row.names = FALSE)
```

이후엔 동일하게 테스트 데이터에 대해 예측하고, count를 한 후 이를 kegg에 올려 결과를 평가받는다.

 예측_배깅_결정트리1.csv 완료(마감일 이후) · 지금	0.90705	0.90705	☐
 예측 dicisiantree1.csv 완료(마감일 이후) · 16분 전	0.91832	0.91832	☐

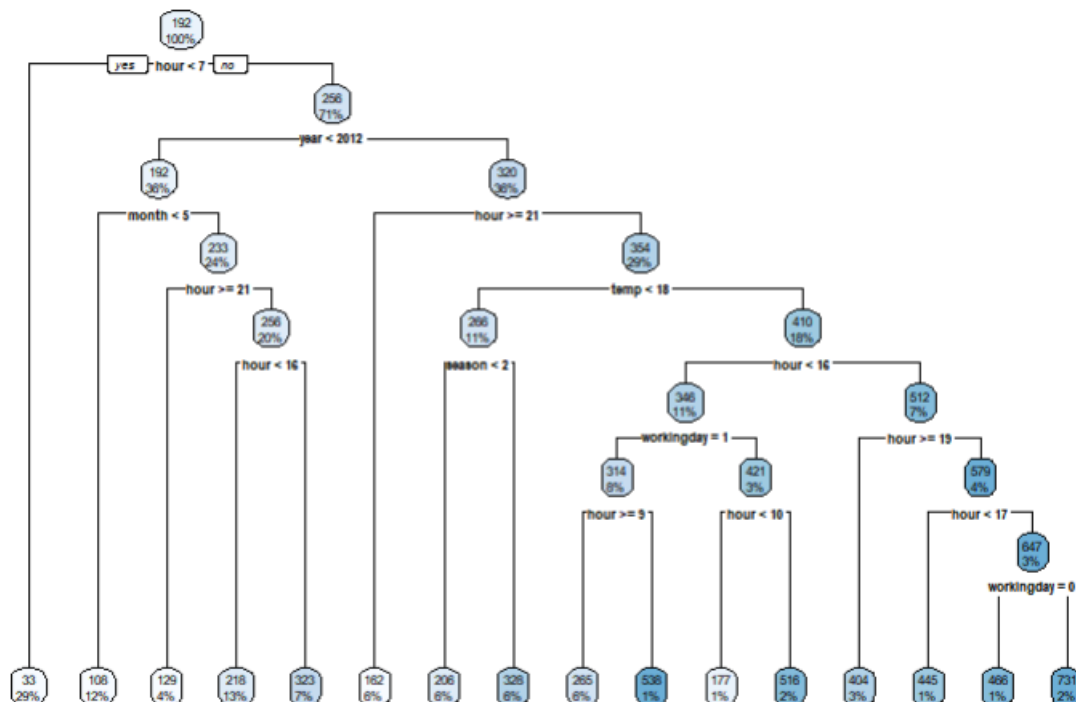
bagging을 사용했지만 오히려 결과가 조금 더 떨어졌다.

그런데 bagging을 사용한 것과, 그렇지 않은 것은 예측 모델링에 어떤 차이가 있을까?

그냥 rpart를 사용한 decision tree 학습과, bagging을 사용한 모델 학습은 단일 decision tree를 사용하여 예측을 수행하는 것과, 여러 개의 decision tree를 사용하여 모델을 학습하는 것의 차이이다.

하나의 decision tree를 사용한다는 것이 무엇인지, 여러개의 decision tree를 사용하는 것은 또 무엇인지 이해가 잘 되지 않아, 이 decision tree를 시각화하여, 어떤 차이가 있을지 직관적으로 보고자 하였다

```
## 시각화
install.packages("rpart.plot")
library(rpart.plot)
rpart.plot(model)
```



이 시각화는 rpart를 사용한 dicision tree이다.

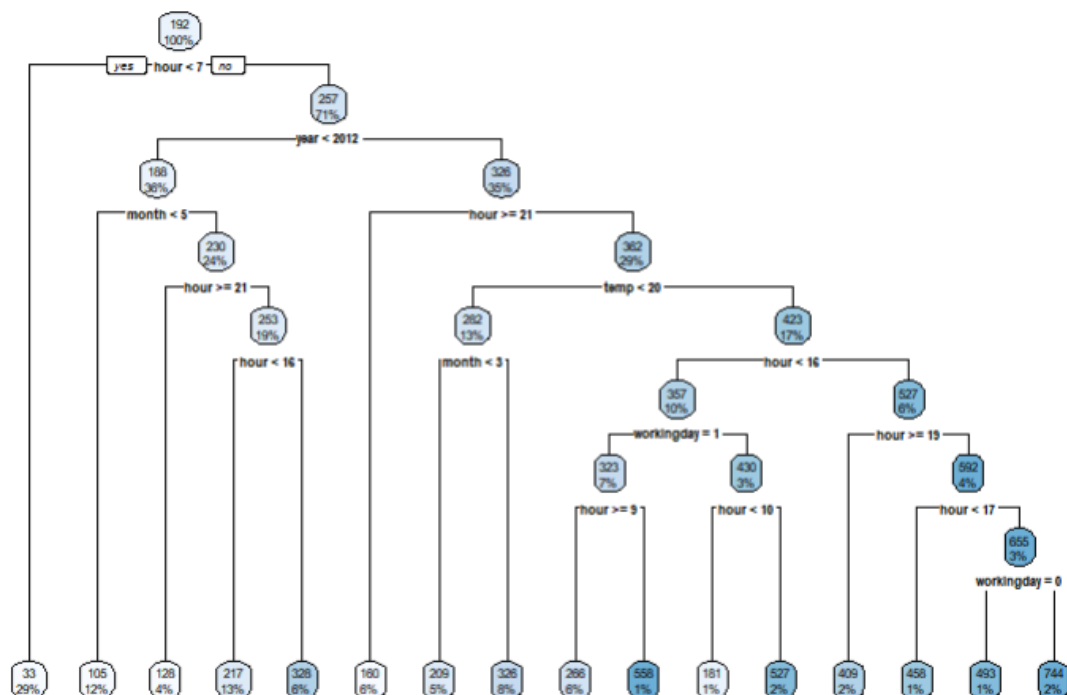
가장 첫 번째 노드는 hour이 7 이상인가 아닌가로 나누어 지며, no일 경우 다시 year이 2012년도 이후인가 아닌가로 분리된다. 이후에는 분기의 여부로 month나 hour,로 분리가 되고, 혹은 team, season, workiingday 등으로 분리된다. 그러나 가장 눈에 띄는 것은 시간에 대한 dicision tree이며, 시간대에 대해 강한 예측이 들어간 것으로 판단할 수 있다.

이번에는 bagging을 적용한 모델에 대해 동일하게 시각화를 진행한 후, 어떤 차이가 있는지 확인해보자

```
# 첫 번째 트리 추출
first_tree <- bagging_model$mtrees[[1]]$btree

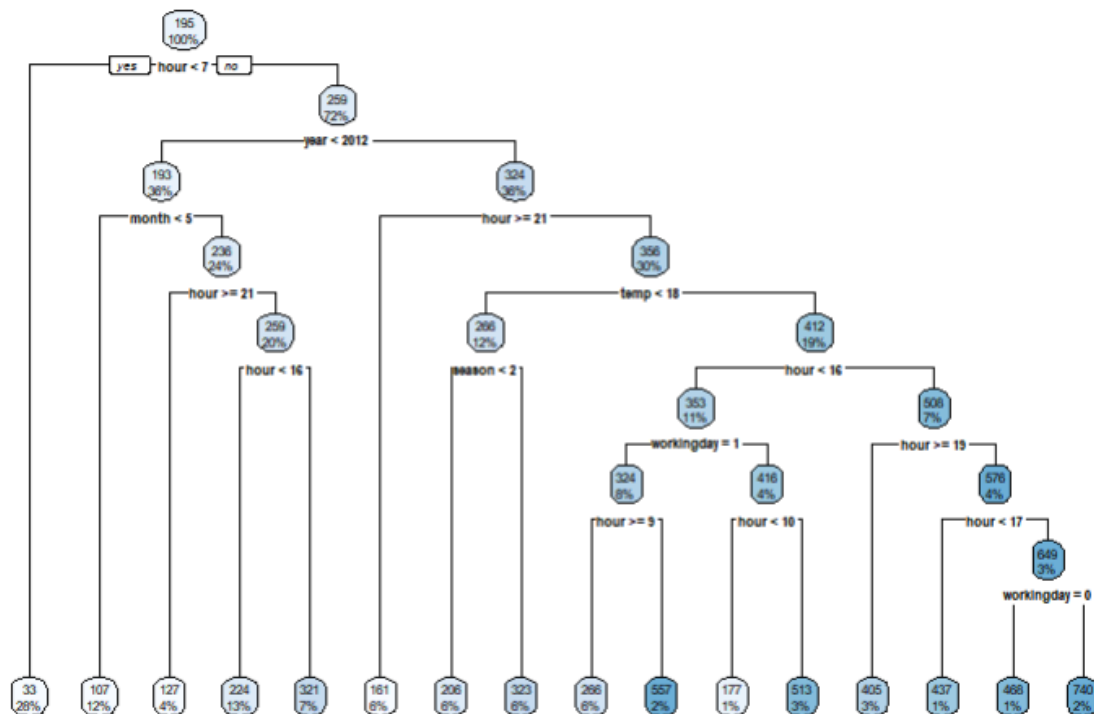
# 첫 번째 트리 시각화
rpart.plot(first_tree)
```

bagging에서는 기본적으로 트리를 25개로 제한을 두었기 때문에, 전체 트리를 시각화하는 것은 어렵다. 대신 n번째 트리를 추출하여 시각화 하는 방식으로 각각의 dicision tree를 확인해 볼 수 있다.



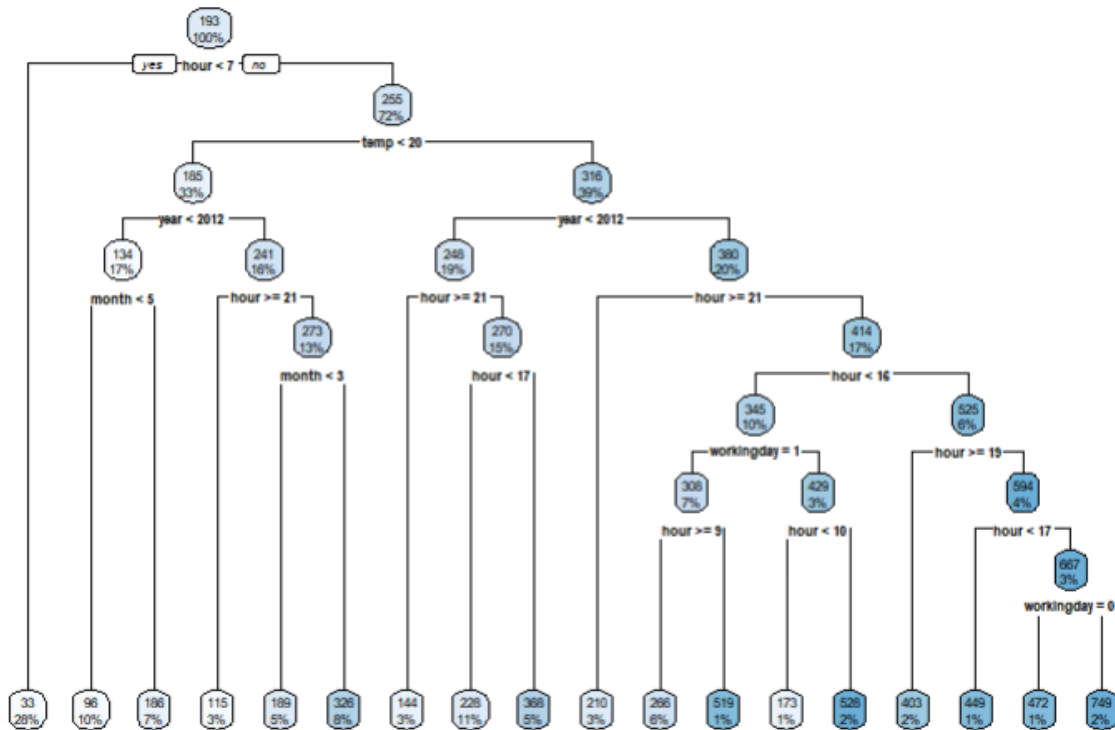
첫 번째 트리는 그 구조는 이전 rpart에서 사용한 트리와 거의 비슷하나, 세세한 값에서 조금씩 차이가 생긴 것 같다.

```
second_tree <- bagging_model$mtrees[[2]]$btree
rpart.plot(second_tree)
```



두 번째 트리 역시, 각 노드의 값이나 비중이 조금씩 달라지기는 하지만 큰 변화가 생기지 않는 것으로 보인다.

```
second_tree <- bagging_model$mtrees[[4]]$btree
rpart.plot(second_tree)
```



네 번째 트리에서 드디어 약간의 구조 변경이 생겼다. 이렇게 다양한 트리가 복합적으로 예측에 영향을 미치는 방식으로 bagging이 작동하게 된다.

Task 3 random forest 모델을 이용해서 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기

- feature importance 계산하여 설명하기

```
# 패키지 설치
install.packages("randomForest")

# 패키지 로드
library(randomForest)
```

random forest모델링을 위한 패키지를 설치하고 로드한다.

```
# Random Forest 모델 학습
set.seed(42) # 재현성을 위해 설정
```



```
rf_model <- randomForest(count ~ ., data = train_data, ntree = 100, importance = TRUE)

# 테스트 데이터에 대한 예측
y_pred_test_rf <- predict(rf_model, X_test)

# 예측 결과를 sample_data에 저장
sample_data$count <- y_pred_test_rf

# CSV 파일로 저장
write.csv(sample_data, "random_forest_submission1.csv", row.names = FALSE)
```

randomForest함수를 통해서, 모델을 학습시킨 후, 예측은 동일하게 수행하고 결과를 kegg에 제출하여 결과를 확인한다.

제출 및 설명	개인 점수 ①	공개 점수 ①	선택된
 랜덤_포레스트_제출1.csv 완료(마감일 이후) · 지금	0.63302	0.63302	☐
 예측_배경_결정트리1.csv 완료(마감일 이후) · 2시간 전	0.90705	0.90705	☐
 예측 dicisiontree1.csv 완료(마감일 이후) · 3시간 전	0.91832	0.91832	☐

랜덤 포레스트의 결과가 오히려 낮게 나온것을 확인할 수 있다.

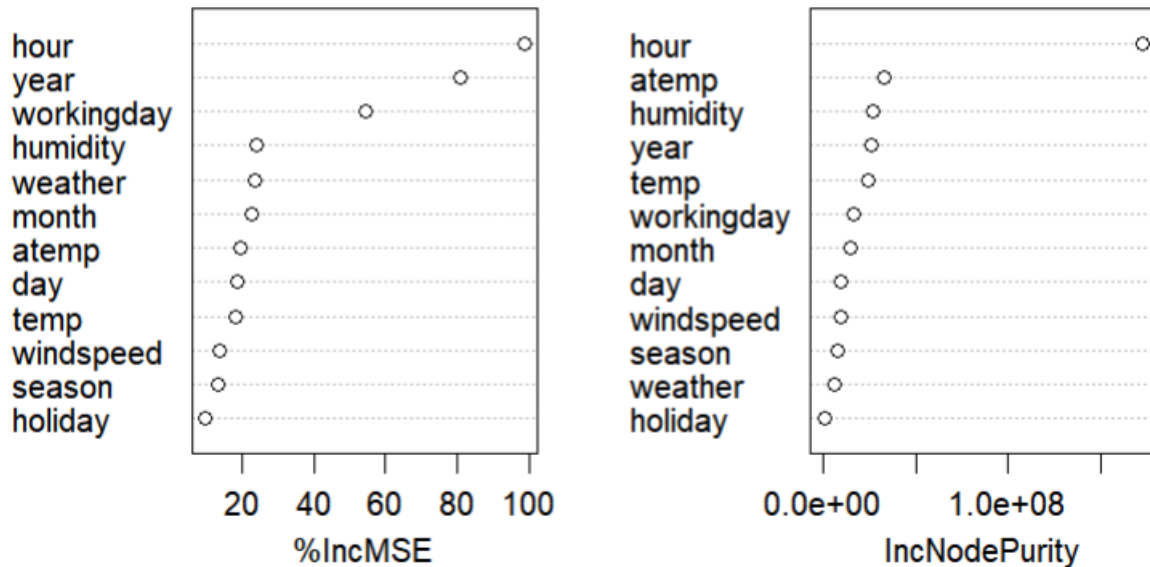
랜덤 포레스트는 피쳐를 자동으로 계산하는데, 어떻게 계산이 되어서 결과가 다소 아쉬운 모습을 보이는지 확인해보자

```
# Feature Importance 계산
importance(rf_model)

# Feature Importance 시각화
varImpPlot(rf_model)
```

importance함수를 통해 모델의 피쳐값을 확인하고, 이를 varImpPlot로 시각화가 가능하다

rf_model



왼쪽 IncMSE는 각 피쳐가 모델 성능에 미치는 영향을 보여주며, 시간, 년도, workingday 변수가 가장 모델 성능에 큰 영향을 미친다. 즉, 해당 변수를 제외하였을때 평균 제곱 오차가 얼마나 증가하는지를 나타낸다.

오른쪽 그래프는 각 피쳐가 트리에서 데이터를 분할할 때 노드 순도(Node Purity)를 얼마나 증가시켰는지를 나타낸다. 즉, 노드 순도라는 값이 클수록 해당 피쳐가 트리의 분할에 큰 영향을 주었다는 것이다. 이를 노드 순도라는 어려운 말 대신 중요도로 치환해도 괜찮을 것 같다.

Task 4 XGBoost를 사용하여 자전거 대여 수량을 예측하고, Kaggle에서 성능 평가하기

```
# XGBoost 패키지 설치
install.packages("xgboost")

# 패키지 로드
library(xgboost)
```

모델링에 필요한 패키지 설치 및 로드

```
# Matrix 형태로 변환
X_train_matrix <- as.matrix(X_train)
X_test_matrix <- as.matrix(X_test)
y_train_matrix <- as.matrix(y_train)
```

XGBoost는 매트릭스를 필요로 하므로, 현재의 데이터를 Matrix형태로 변환한다

```
# XGBoost 모델 학습
dtrain <- xgb.DMatrix(data = X_train_matrix, label = y_train_matrix)
dtest <- xgb.DMatrix(data = X_test_matrix)
```

XGBoost는 매트릭스나 데이터 프레임 형태의 데이터를 DMatrix라는 새로운 형태로 받아 들여 사용한다. 이를 위해 xgb.DMatrix함수를 통해서 데이터를 DMatrix의 형태로 변환한다.

```
params <- list(
  objective = "reg:squarederror", # 회귀 문제이므로 제곱 오차 사용
  eta = 0.1, # 학습률
  max_depth = 9, # 트리 깊이
  subsample = 0.8, # 데이터 샘플 비율
  colsample_bytree = 1.0 # 피쳐 샘플링 비율
)
```

params는 파라미터를 정의한 내용이다.

가장 먼저 objective는 어떤 방법으로 문제를 해결하며 예측할 것인가에 대한 내용으로, 자전거의 대여를 예측하는 문제는 회귀 문제이므로 제곱오차를 줄이는 방법으로 예측을 진행하고자 하였고, 때문에 reg:squarederror 를 사용하였다

다음 `eta`는 학습률을 의미하며 새로 학습된 모델이 기존 모델에 얼마나 영향을 미칠지를 결정한다

0.1은 비교적 낮은 학습률로, 모델이 학습되는 속도는 느리지만 대신 과적합을 방지할 수 있다.

`max_depth`는 트리의 깊이를 설정하는 것으로, 이진 트리에서 몇 번째 깊이까지를 수행할지 결정하는 파라미터이다.

`subsample`는 데이터 안에서 또 샘플의 비율을 설정하는 파라미터로, 우리는 결국 패턴을 찾아야 하며 그를 위해 모든 데이터를 사용할 필요는 없기 때문에 그 중에서도 80%만 사용하게 되었다.

`colsample_bytree` 이는 피처 샘플링을 하는 비율을 의미하며 나는 모든 피처를 사용하게 하기 위해 1로 설정하였다.

```
# 모델 학습
xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 100, # 트리 생성 수
  watchlist = list(train = dtrain),
  verbose = 1
)
```

다음은 모델 학습으로, `xgb.train`을 통해 모델을 학습하는데, 파라미터는 이전에 지정한 `params`를 사용하고, 데이터는 `dtrain` 트리 생성 수는 100개로 제한한다. 트리를 거듭할수록 모델의 성능이 좋아지겠지만, 이는 과적합의 위험도 함께 가지기에 일단은 100으로 제한한다.

```
# 테스트 데이터에 대한 예측
y_pred_test_xgb <- predict(xgb_model, dtest)
```

```
# 예측 결과를 sample_data에 저장
sample_data$count <- y_pred_test_xgb

# CSV 파일로 저장
write.csv(sample_data, "xgboost_submission1.csv", row.names
= FALSE)
```

이후 동일하게 예측값을 업로드한다

The screenshot shows the Kaggle competition interface. On the left, a list of submissions is visible, including 'xgboost_submission1.csv' (uploaded 2m ago), 'xgboost_submission1.csv' (uploaded 19m ago), '랜덤_포레스트_제출1.csv' (completed 43m ago), '예측_배깅_결정트리1.csv' (completed 3h ago), and '예측 dicisiontree1.csv' (completed 3h ago). On the right, the details for 'xgboost_submission1.csv' are shown, indicating it was uploaded 19 minutes ago. Below the submission name, a list of errors is displayed, each with a row number and a message: '오류: 값 '-1.05041027069092'가 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(4번째 줄, 23', '오류: 값 '-1.21363508701324'가 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(5번째 줄, 23', '오류: 값 '-1.67730379104614'가 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(6번째 줄, 23', '오류: 값 '-3.76239919662476'은 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(29행, 23', '오류: 값 '-0.264294266700745'가 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(30행, 23', '오류: 값 '-0.291567653417587'은 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(54행, 23', '오류: 값 '-23.6487159729004'가 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(56행, 23', '오류: 값 '-2.61852598190308'은 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(77행, 23', '오류: 값 '-0.517043113708496'은 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(78행, 23', '오류: 값 '-30.0630931854248'은 필수 값 세트 'Non-negative = [0, ∞)'에 없습니다(79행, 23'. Below the errors, there is a section for '업로드된 파일' (Uploaded files) showing 'xgboost_submission1.csv (253키비)' and a '설명' (Description) field with the placeholder text '설명을 입력하세요'.

그런데 다음과 같이 오류가 발생하였다. 자전거의 예측은 음수가 나올 수 없는데, 모델이 이를 음수로 예측하였다.

우선은 지금 모델 자체의 성능평가를 위해, 해당 오류가 나온 음수 값들은 전부 0으로 바꾸는 후처리를 진행하고자 한다

```
# 예측값이 음수일 경우 0으로 변환
y_pred_test_xgb <- pmax(y_pred_test_xgb, 0)
# 예측 결과를 sample_data에 저장
sample_data$count <- y_pred_test_xgb
```

예측값이 0 이하일 경우, 전부 0으로 바꾸었다.

제출 및 설명	개인 점수 ①	공개 점수 ①	선택됨
 xgboost_submission1.csv 완료(마감일 이후) · 지금	0.64525	0.64525	☐

이제 결과를 확인하니, 0.64점 정도의 결과가 나왔다.